

Introduction to Debugging



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

**Physics
Department**
Ph.D school

**Dept. of
Math., Inf.
and Geosciences**



Luca
Tornatore
INAF

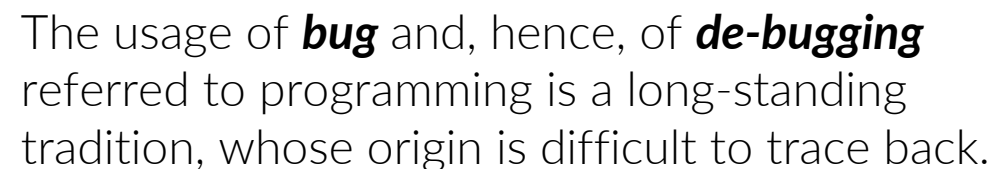


Outline



- ## Debugging
- *Serial codes*
 - *general concepts*
 - *using gdb*
 - *using sanitizers*
 - *OpenMP codes*
 - *MPI codes*

- ## Profiling
- *Using valgrind*
 - *Using perf*
 - *Instrumentation*
 - *Parallel codes*



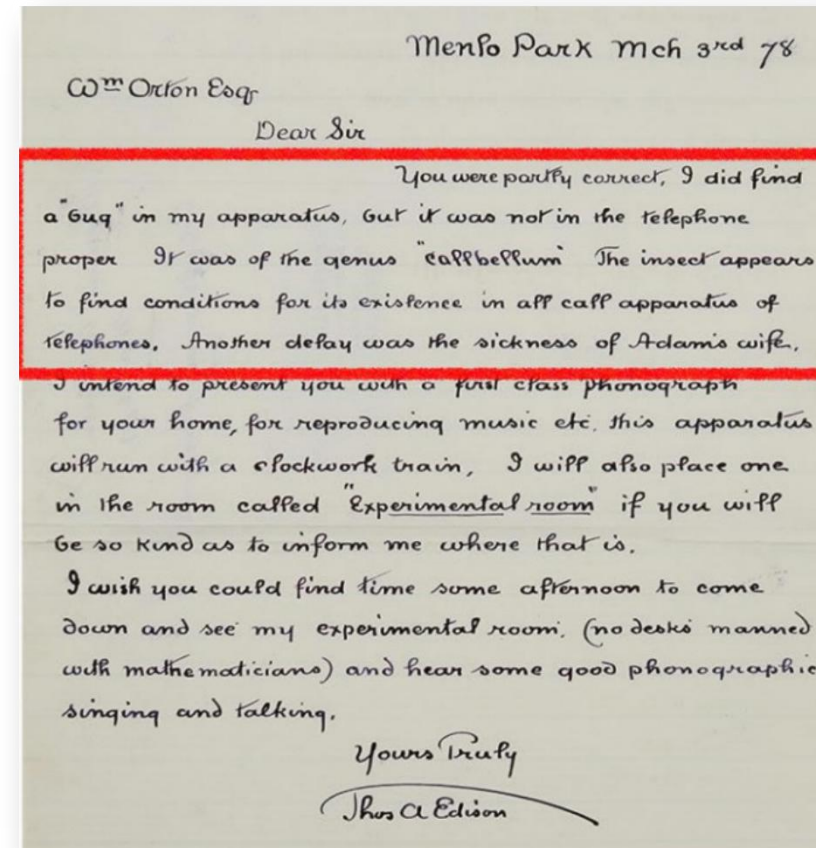
Adm. Grace Hopper is reported to have written in
its diary about that as
“first *actual* case of bug being found”

Introduction (*warm-up*)

However, **Thomas Edison** found another *actual* bug in one of his phones, as he reports in a letter to an associate.

He later writes:

"It has been just so in all of my inventions. The **first step** is an intuition and comes with a burst, **then difficulties arise**—this thing gives out and [it is] then that "**bugs**"—as such little faults and difficulties are called—show themselves and **months of intense watching, study and labor are requisite** before commercial success or failure is certainly reached."



Introduction (*warm-up*)

It is difficult to trace back how this term has been poured into computer programming jargon. However, already in the early 60s it was appearing in technical papers without need of explanation.

The immortal I. Asimov used it in a 1944 short robot story “Catch that rabbit”, and his incredible influence contributed much to make the term popular.

More funny infos:

[IEEE Annals of the History of Computing](#)
(Volume: 20 , [Issue: 4](#) , Oct-Dec 1998)

Stalking the Elusive Computer Bug

PEGGY ALDRICH KIDWELL

From at least the time of Thomas Edison, U.S. engineers have used the word “bug” to refer to flaws in the systems they developed. This short word conveniently covered a multitude of possible problems. It also suggested that difficulties were small and could be easily corrected. IBM engineers who installed the ASSC Mark I at Harvard University in 1944 taught the phrase to the staff there. Grace Murray Hopper used the word with particular enthusiasm in documents relating to her work. In 1947, when technicians building the Mark II computer at Harvard discovered a moth in one of the relays, they saved it as the first actual case of a bug being found. In the early 1950s, the terms “bug” and “debug,” as applied to computers and computer programs, began to appear not only in computer documentation but even in the popular press.

Introduction

Stalking computer bugs—that is to say, finding errors in computer hardware and software—occupies and has occupied much of the time and ingenuity of the people who design, build, program, and use computers.¹ Early programmers realized this with some distress. Maurice Wilkes recalls that in about June of 1949:

I was trying to get working my first non-trivial program, which was one for the numerical integration of Airy’s dif-

published in the *Annals* in 1981. Here the time is given as the summer of 1945, but the computer is the Mark II, not the Mark I. A photograph shows the moth taped in the logbook, labeled “first actual case of bug being found.” Small problems with computers have been called bugs ever since.⁵

1/2

Introduction

The quote from Edison might be too dramatic and emphatic, but the real point is: *de-bugging* is an inherent and intrinsic activity in software development.

So, provided that

- you'll have close encounters with bugs in your life;
- de-bugging is a fundamental and unavoidable part of your work;

what is the best way to proceed once you observe “a bug”?

The symptoms (*serial codes*)

1) CRASH

*segfault, abort, *** stack smashing detected ****

Program terminates abnormally with a signal like the ones above.

Usually: memory error, NULL dereference, stack overflow, infinite recursion, or library-detected corruption (e.g. malloc/free invariants violated).

2) WRONG ANSWER

silent – no signal, no exit code

Program completes, but the output is incorrect.

Usually: logic error, undefined behaviour, numerical drift, or uninitialized memory read.

3) HANG / DEADLOCK

program does not progress, does not terminate

Usually (serial): infinite loop, blocked on I/O.

Usually (parallel): lock cycle, missed wake-up, MPI receive with no matching send.

4) PERFORMANCE DEGRADATION

correct, but unexpectedly slow

Algorithmic mistake, cache pathology, lock contention, oversubscription. Strictly speaking this is more profiling related, however if your performance degrades after some change that might be due to some bug.

The typical causes

A) MEMORY ERRORS

out-of-bounds, use-after-free, double-free, uninit reads, leaks

Pointers corruption; heap corruption.

The symptom often appears far from the cause — heap corruption in one allocation may not crash until a later `free()` touches the corrupted metadata.

Tools: GDB, ASan, Valgrind Memcheck.

B) UNDEFINED BEHAVIOUR

signed overflow, strict aliasing, division by zero, alignment, type punning

That may appear with optimization, since the optimizer is allowed to assume UB does not occur, so your code becomes nonsense with `-Ox`.

Tools: UBSan, compiler warnings, static analysis.

C) LOGIC ERRORS

off-by-one, wrong comparison, wrong base case, wrong formula

The execution itself is correct; but there are some clerical errors.

Tools: GDB might be very useful to follow execution step-by-step

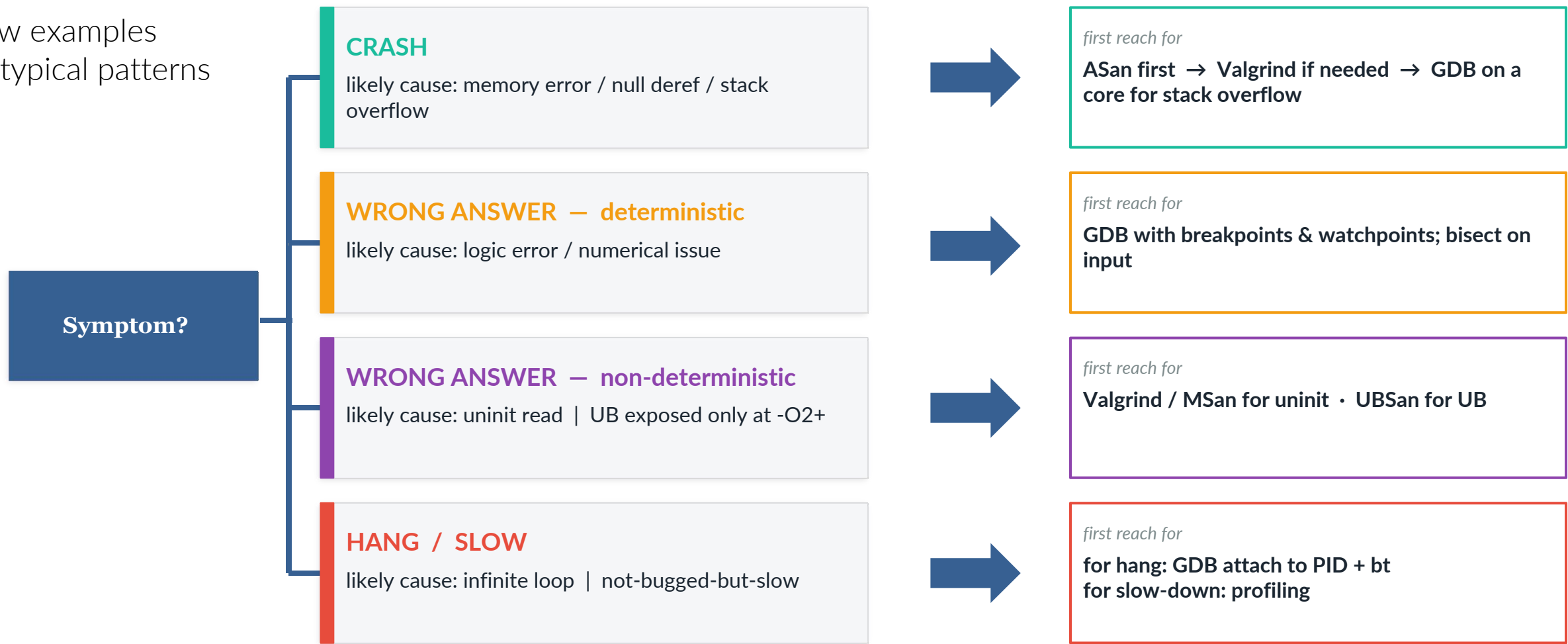
D) NUMERICAL DRIFT

catastrophic cancellation, loss of significance, NaN propagation

Not always a bug, strictly speaking.

Tools: re-do the analytical work, condition-number analysis, check whether precision problems arises.

Few examples of typical patterns



Small vocabulary

ASan: Address Sanitizer; MSan: Memory Sanitizer; UB: Undefined Behaviour; bt: backtrace

Be pre-emptive

Compiler's Warnings

```
-Wall -Wextra -Wpedantic
-Wshadow -Wconversion
-Wnull-dereference -Werror
```

be as strict as possible
*If your code does not compile cleanly at this level, fix it before doing anything else. Each warning is a potential bug; **-Werror** pushes you more.*

Assertions

```
#include <assert.h>
assert(ptr != NULL);
assert(idx >= 0 &&
        idx < N);
```

Enrich your code with **assert(..)** instead of comments about what should be true.
 The cost is zero with -DNDEBUG builds.

This catches (some) bugs at the point of violation.

Static Analysis

```
clang-tidy --checks='*'
cppcheck --enable=all
clang --analyze
```

Catches dead code, possible null derefs, unused values, type confusion.
 Especially useful in CI / pre-commit.

Hardening builds

```
-D_FORTIFY_SOURCE= 1|2|3
-fstack-protector-strong
-fstack-clash-protection
```

Adds compile-time and runtime checks for the obvious memory-handling routines.
 Add for new builds / changes with the same optimization flags used for production.

Be pre-emptive

`_FORTIFY_SOURCE` (since glibc 2.3.4)

Defining this macro causes some lightweight checks to be performed to detect some buffer overflow errors when employing various string and memory manipulation functions (for example, `memcpy(3)`, `memset(3)`, `strcpy(3)`, `strncpy(3)`, `strcat(3)`, `strncat(3)`, `sprintf(3)`, `snprintf(3)`, `vsprintf(3)`, `vsnprintf(3)`, `gets(3)`, and wide character variants thereof). For some functions, argument consistency is checked; for example, a check is made that `open(2)` has been supplied with a `mode` argument when the specified flags include `O_CREAT`. Not all problems are detected, just some common cases.

If **`_FORTIFY_SOURCE`** is set to 1, with compiler optimization level 1 (`gcc -O1`) and above, checks that shouldn't change the behavior of conforming programs are performed. With **`_FORTIFY_SOURCE`** set to 2, some more checking is added, but some conforming programs might fail.

Some of the checks can be performed at compile time (via macros logic implemented in header files), and result in compiler warnings; other checks take place at run time, and result in a run-time error if the check fails.

With **`_FORTIFY_SOURCE`** set to 3, additional checking is added to intercept some function calls used with an argument of variable size where the compiler can deduce an upper bound for its value. For example, a program where `malloc(3)`'s size argument is variable can now be fortified.

Use of this macro requires compiler support, available since gcc 4.0 and clang 2.6. Use of **`_FORTIFY_SOURCE`** set to 3 requires gcc 12.0 or later, or clang 9.0 or later, in conjunction with glibc 2.33 or later.

Additional practical advices

Anti-patterns to watch

The bug disappears when adding a printf

likely: memory error whose effect depends on stack / heap layout – the printf shifted things; the bug is still there.

The bug vanishes at -O0

likely: almost certainly UB. The optimizer relied on the no-UB assumption; at -O0 the literal code happened to work.

“It works on my machine”

likely: different memory layout (heap, stack, ASLR), libc version, OMP/MPI environment, compiler version.
Always test on different platforms.

Instrumentation appears to fix the bug

likely: ASan/UBSan changes allocation patterns and zero-init; the symptom hides while the bug remains. Typical for memory errors. Try different inputs.

What if..

Variable shown as <optimized out> in GDB

likely: compile with -Og, mark the variable volatile temporarily, or watch the memory location instead of the symbolic name.

The bug only reproduces with the full input set

likely: you have not minimized enough. Bisect the input. Most bugs survive a 100× reduction.

Compiling for debugging

-g / -g3

Embed debug info (DWARF). **-g3** also records macro definitions, so **print MYMACRO** works inside gdb. The cost is binary size, never speed.

-ggdb / -ggdb3

Same idea, but emit the richest format gdb understands instead of the platform default. On Linux the difference is small; harmless to prefer.

-O0 vs -Og

-O0: no optimization, every variable alive, every line steppable. **-Og**: the debug-friendly subset of **-O1** — faster code, still honest line info. A good daily default.

-fno-omit-frame-pointer

Keep **%rbp** as frame pointer. DWARF unwind tables are slow to walk and sometimes incomplete; frame pointers make stack walks cheap — sampling profilers (**perf**) depend on them.

-fno-inline

When you really need to step **into** calls: an inlined function has no frame to break on, and gdb can only pretend. Pair with **-Og** when **-O0** is too slow.

-fsanitize=...

See later for details

Production builds: keep the debug info, just not in the binary. Build optimized **with** **-g3**, split the debug info aside, ship the stripped binary and archive the **.debug** file. A core dump plus that archive is a fully debuggable session.

```
gcc -O2 -g3 -o prog ...  
objcopy --only-keep-debug prog prog.debug  
objcopy --strip-debug prog  
objcopy --add-gnu-debuglink=prog.debug prog
```


Starting gdb

```
gdb ./prog
```

```
(gdb) set args 1 2  
(gdb) run
```

The plain way. Arguments are set inside the session — convenient when you want to vary them between runs without leaving gdb.

```
gdb --args ./prog 1 2
```

```
(gdb) run
```

Arguments on the command line, once. Practical when args are fixed; **run** is then just run.

```
gdb -tui ./prog
```

```
source panel visible
```

The same session with more glass: a curses source window from the start.
See later for some options

```
gdb -p PID
```

```
attach to a live process
```

No restart: gdb grabs a process already running.

~/.gdbinit is read at every startup: command history, pretty-printing defaults, your own command definitions.

Some more details later

Leaving and interrupting.

quit (or Ctrl-D) ends the session;

Ctrl-C does **not** kill gdb — it interrupts the program under debug and gives you the prompt back, with the program suspended wherever it was.

Controlling the flow

command	-	description
---------	---	-------------

line by line

next	n	execute the next source line; calls are run through, not entered.
step	s	execute the next source line, descending into calls (with debug info).
nexti / stepi		the same two, one machine instruction at a time.

larger strides

continue	c	run until the next breakpoint, signal, or exit.
finish	fin	run until the current function returns — and print the return value.
until	u	run until a source line beyond the current one in this frame: the civilized exit from a loop you are tired of stepping.
advance LOC		run to LOC and stop there, like a breakpoint that exists for one trip only.

- get the habit of using short versions: **n**, **s**, **c**, **u**, **bt**, ...
- **enter** repeats the previous command
- some commands admits arguments; for instance
n COUNT
s COUNT
c COUNT,

skip COUNT events

Controlling the flow

command	-	description
---------	---	-------------

line by line

next	n	execute the next source line; calls are run through, not entered.
step	s	execute the next source line, descending into calls (with debug info).
nexti / stepi		the same two, one machine instruction at a time.

larger strides

continue	c	run until the next breakpoint, signal, or exit.
----------	---	---

- get the habit of using short versions: **n**, **s**, **c**, **u**, **bt**, ...

- **enter** repeats the previous command

- some commands admits arguments; for instance
n COUNT
s COUNT
c COUNT

Note for the multi-threaded codes

set scheduler-locking on|step|off — controls which threads are allowed to run while you step.

Breakpoints

Defining

```
break find_max_recursive
```

by function

```
break gdb_playground.c:142
```

by file:line

```
break +5      break -3
```

relative to where you stand

```
break find_kth if k > 2
```

conditional — the predicate
is re-evaluated at every hit

```
tbreak main
```

one-shot: deletes itself
after the first hit

Housekeeping

info breakpoints list them, numbered;

disable *n* · **enable** *n* · **delete** *n* · **clear** *LOC* act on those numbers
(or on a location).

Attaching commands: the do-it-yourself tracepoint

```
(gdb) break find_max_recursive
Breakpoint 1 at 0x...: line 131.
(gdb) commands 1
> silent
> print *head
> continue
> end
```

Every hit now prints the node and moves on: a log of the recursion
without touching the source. **silent** suppresses the usual stop chatter.

Inspecting variables

Variables and expressions

print **EXPR** (**p**) evaluates any C expression in the current frame — arithmetic, casts, dereferences, even function calls in your program. A format letter changes the rendering: **p/x** **flags**.

/x hex**/d** signed**/u** unsigned**/o** octal**/t** binary**/f** float**/c** char**/s** string

Raw memory: the x command

x/4dw &array**N — repeat count**

how many units

f — format

same letters as print

u — unit sizeb=1 h=2 w=4 g=8
bytes

Always confusing the first time; the order is **count, format, unit**. So **x/8xb p** is eight bytes in hex from **p**; **x/s str** reads a NUL-terminated string; **x/4i \$rip** disassembles the next four instructions.

Frame and machine

info locals every local of the current frame**info args** the arguments the frame was called with**info registers** the CPU state; then **p \$rax**, **p/x \$rip** address them individually

- **display c->value** re-evaluates and prints the expression at **every stop** — ideal while stepping a loop.
- **info display** lists them
- **undisplay N** removes one.

Set **set print pretty on** once (or in **~/.gdbinit**) and printing structs will go one member per line instead of a single unreadable row.

Inspecting the stack

Most often used commands

backtrace (bt)	the call chain, innermost frame first.
bt full	the same, plus every local of every frame — the first command at any crash, and on any core file.
bt 10 · bt -10	the first ten frames, or the last ten — for when the chain is thousands frames long (i.e. recursion).
frame N · up · down	move between frames; print and info locals then act on the selected one.
info frame	the anatomy of the selected frame: return address, saved registers, frame base.

thread apply all bt

A backtrace of **every** thread. Useful with threads: it helps diagnosing most deadlocks — who waits on what, frozen mid-call.

“Backtrace stopped: previous frame identical to this frame”

gdb's way of telling you the recursion ran away. **2_stack_overflow.c** dies exactly like this: a logically correct function, 8 MB of default stack (**ulimit -s**), and a SIGSEGV on the guard page. A bare segfault is not always a pointer bug; sometimes it is arithmetic done in stack frames.

First warm-up with gdb

```
$ gdb --args ./gdb_playground \  
    1.0 2.0 3.0 4.0 5.0  
  
...  
(gdb) list main  
(gdb) break main  
Breakpoint 1 at 0x...: line 161.  
(gdb) run  
(gdb) next  
(gdb) next  
(gdb) print head  
$1 = (Node *) 0x55555555a2a0  
(gdb) print *head  
$2 = {id = 1, value = 1, next = 0x0}  
(gdb) continue
```

list main

show the source around a function — it takes **file:line** too.

break main · run

the program starts and stops on the first executable line of **main**;

next

one source line at a time; calls are executed but not entered (**step** is the entering variant).

print head · *head

the pointer, then the struct it points to. gdb reads your types from the DWARF info — this is what **-g3** provides.

continue

run to the next breakpoint, or to the end.

Use **gdb_playground.c** with five values
the very run that exposes bug 1 in **count_in_range** (**exercises.pdf**, exercise 1).

Watchpoints

When some memory error is involved, for instance an out-of-bound write, it might be difficult to get when a variable is overwritten - which is the symptom. That, in fact, typically does not happen where explicitly the variable is accessed. For this cases, *watching* the variable, or the memory location, is a good tool.

```
watch k
```

stop whenever k is **written** —
by anyone, anywhere.

```
rwatch k
```

stop whenever k is **read**.

```
awatch k
```

either way.

```
watch -l EXPR
```

watch the **address** EXPR lives
at, not the symbolic
expression — survives scope
changes.

```
watch -l *p mask 0xfffff000
```

watch a whole aligned region
through a bitmask.

In practice — exercises.pdf, exercise 2: enter `find_kth`, then `watch k`. Two stops later the pre-decrement off-by-one is in plain sight, with no stepping through iterations. When a loop ends one trip early or late, watch the counter, not the code.

Remind:

x86-64 has exactly **four debug registers** (DR0–DR3): four hardware watchpoints, **in total**. One past that limit, gdb falls back — silently, except for the wording — to **software watchpoints**: it single-steps the entire program and re-checks the value after **every instruction**.

The slowdown is orders of magnitude. For one loop counter the cost is negligible; for a large struct it is geological.

You can tell which kind you got from the announcement:

“**Hardware watchpoint 2**” vs “**Watchpoint 2**”.

Watching a 64-byte struct needs more than the registers offer; watching `&s->field` with `watch -l` usually fits.

Catchpoints

A breakpoint names a **place**; a catchpoint names a **class of events**. It is the right tool precisely when you do not yet know where in the code the event originates — which, for a SIGSEGV, is the normal situation.

```
catch signal SIGSEGV
```

stop at the instant the signal is delivered — before the process dies of it.

```
catch signal all
```

any signal: also the polite ones (SIGTERM, SIGUSR1) your batch system sends.

```
catch throw
```

C++: stop at the throw, not wherever the exception finally lands.

```
catch syscall openat
```

stop at a specific system call — “who keeps opening this file?”

```
catch fork
```

stop when the process forks; decide there which side to follow.

handle [signal] [actions] configure how gdb reacts to signals; **catch** decides **where gdb stops**, **handle** decides **what gdb does** with signals not caught.

info signals reports about the signals lookup table

gdb -tui

built in

Ctrl-X A toggles it in any session; **layout**
src · asm · split · regs choose the panels;
Ctrl-X 2 cycles a second window; **Ctrl-L** redraws after a resize

gdb-dashboard

github.com/cyrus-and/gdb-dashboard

A single Python file that becomes your **.gdbinit**: modular panels (source, assembly, stack, memory, registers) redrawn at every stop, each configurable or removable.

gdbgui

gdbgui.com · `pip install gdbgui`

A browser front-end over gdb: source, variables, and the call stack as a web page, point and click.

seergdb

github.com/epasveer/seer

A gui frontend to gdb for Linux.
Includes a graphical arrays inspections, memory visualizer, structur visualizer

Post-mortem

1

```
ulimit -c unlimited
```

Per shell — and per **job script**: most systems default to 0 and silently does not produce the core.

2

```
cat /proc/sys/kernel/core_pattern
```

Decides where cores are written. On systemd machines they go to the journal: **coredumpctl list**, then **coredumpctl gdb**.

3

```
gdb ./prog core
```

(or **-c core**) The binary must be **the one that crashed**, with its debug info — the **.debug**-file workflow we have mentioned before is produced for this reason.

4

```
bt full
```

The first move on any core; then **frame**, **print**, as in a live session. You inspect, not resurrect: there is no process to **continue**.

Debugging live processes

```
$ pidof prog          # or: top, ps
$ gdb -p 41312
(gdb) bt
(gdb) detach          # it keeps running
```

Attaching **suspends** the process while you inspect it; **detach** releases it, alive, exactly where it was.

Useful in serial for a hung program you would rather not restart, or a long computation whose state you want to peek at without sacrificing it — attach, look, detach.

Control who may attach to whom: `/proc/sys/kernel/yama/ptrace_scope`

- 0** classic: any process of your own UID.
- 1** restricted: only your descendants — the default on most distros; `gdb -p` on an unrelated PID fails with “Operation not permitted”.
- 2** admin only: `CAP_SYS_PTRACE` required.
- 3** no attach at all; the hardened setting, immutable until reboot.

Seize the moment

rr-project.org — Mozilla's record-and-replay debugger

```
$ rr record ./prog args...
$ rr replay
(rr) continue           # to the crash
(rr) watch -l corrupted_var
(rr) reverse-continue    # backwards!
(rr) reverse-step
```

rr-project.org

example usage

- 1) run forward** to the corrupted value, set a watchpoint on it
- 2) run backwards** — execution stops at the write that did the damage.

What it does. Records every source of non-determinism — syscall results, signals, thread interleavings — once. Replay is then bit-for-bit deterministic, every time, which is what makes reverse execution actually work.

When to use. The bug that fires one run in five. Record until a run fails: that recording now fails 100% of the time, and you debug it at leisure — forever, and in both directions. (**rr record --chaos** randomizes scheduling to make rare race conditions happening sooner).

Limits

- Linux only; x86-64 mature, AArch64 more recent.
- Needs hardware performance counters — often unavailable inside VMs and containers; try it before you rely on it.
- Records a whole process tree, but replays threads serialized on one core: ~1.2–2× for serial code, much more for parallel hot loops.
- Hardware that bypasses the kernel (GPU APIs, some MPI/RDMA fast paths) is invisible to it.
- Traces live on disk and grow with run length.

Compiler-instrumented binaries that watch themselves run. You build once with a **-fsanitize=...** flag; the resulting executable carries the instrumentation for a class of bug, and tells you — in almost plain English, with a stack trace — the instant the bug fires.



The different Sanitizers

ASan	AddressSanitizer	memory: heap and stack overflows, use-after-free, double-free, leaks (with LSan)
UBSan	UndefinedBehavior	signed overflow, alignment violations, shift overflow, bad casts, null deref
MSan	MemorySanitizer	reads of uninitialised memory — bit by bit. Clang only.
LSan	LeakSanitizer	allocations that no live pointer reaches by program exit. Bundled with ASan.
TSan	ThreadSanitizer	data races in multi-threaded code. We meet it again in Section D.

The address sanitizers

```
gcc -O1 -g3 -fno-omit-frame-pointer -fsanitize=address ...
```

What it catches

- buffer overflows — heap, stack, globals — read or write
- use-after-free, use-after-return, use-after-scope
- double-free, invalid free, mismatched new/free or malloc/delete
- some intra-object overflows, with the right compile flag
- memory leaks at exit

How it works

A **shadow map**: one byte of shadow memory describes the addressability of every eight application bytes (0 = all valid, 1–7 = first N valid, negative codes for stack-freed / heap-freed / redzone).

Every load and store is instrumented to check its shadow byte first; **redzones** — poisoned bytes inserted around every allocation — turn an off-by-one write into a deterministic stop, at the write itself, not at the consequence.

Cost. About 2× CPU, 3× memory. Acceptable for development, not in real production.
Stack frames grow because of redzones; the heap grows because every allocation is padded.

the strcpy on line 93 wrote past the buffer malloc'd on line 89 — same shape for 3_use_after_free.c, 6_double_free.c, 7_oob_read.c

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x...10 at pc 0x...
```

```
WRITE of size 17 at 0x603000000010 thread T0
```

```
#0 0x... in set_user_name 1_heap_overflow_delayed.c:93
```

```
#1 0x... in main 1_heap_overflow_delayed.c:126
```

```
0x603000000010 is located 0 bytes to the right of 16-byte region [0x...000,0x...010)
```

```
allocated by thread T0 here:
```

```
#0 0x... in malloc
```

```
#1 0x... in set_user_name 1_heap_overflow_delayed.c:89
```

The UB sanitizers

```
gcc -O1 -g3 -fsanitize=undefined ...
```

Compile-inserted runtime checks for forms of UB that in C standard can not happen.

Cost is typically a few percent — so it is reasonable to leave on in production to verify integrity of representative runs.

What it catches

- signed integer overflow
- alignment violations — misaligned pointers, hard fault on ARM
- shift count out of bounds, or shifting into the sign bit
- integer divide by zero, `INT_MIN / -1`
- null-pointer arithmetic and dereference
- pointer arithmetic past the end of an array (where known)
- bool, enum values outside their declared range
- function called through a pointer of incompatible type

Demo A from `8_signed_overflow.c`

```
int i = INT_MAX;  
i = i + 1;    /* UB */
```

what UBSan prints, on the line of the addition

```
8_signed_overflow.c:107:9: runtime error:  
signed integer overflow:  
2147483647 + 1 cannot be represented  
in type 'int'
```

8_signed_overflow.c, demo B: a loop whose exit relies on wraparound. At **-O0** it runs slowly to the end; at **-O2** the optimizer reasons that signed overflow is UB, therefore **`i > 0`** is always true, therefore the loop is infinite — and it removes the exit branch entirely. Remind that optimizer *assumes* that no UB happens: that is why using sanitizers regularly is useful to ensure correctness.

The Memory and Leaks sanitizers

MemorySanitizer (MSan)

Clang only — gcc has not implemented it.

```
clang -O1 -g3 -fsanitize=memory ...
```

Tracks the initialisation state of **every bit** of memory and flags reads that depend on uninitialised data. Catches the subtle 'initialised on some paths only' pattern that escapes both -Wuninitialized and casual testing.

Watch out False positives unless every library — libc included — is itself MSan-instrumented. Manageable on own-source projects; awkward on HPC stacks.

exercise: **4_uninitialized_read.c** — the **flag=0** path leaves **offset** undefined; MSan reports the use site and the declaration site of the local.

LeakSanitizer (LSan)

Bundled with ASan; or standalone with -fsanitize=leak.

```
gcc -O1 -g3 -fsanitize=address ... # leaks included
```

At program exit, walks the heap and reports every block that no live pointer can reach, with the full stack trace at the **malloc** that allocated it. Standalone **-fsanitize=leak** is much cheaper than the full ASan if leaks are the only concern.

Compared to Valgrind. LSan reports one class (leaked). Valgrind's Memcheck subdivides: definitely lost / possibly lost / still reachable — sometimes useful, often more than you need.

exercise: **5_memory_leak.c** — the realistic shape: a leak that lives only on the error path, invisible in happy-path tests, fatal in week-long runs at scale.

The threads sanitizers

TSan instruments every memory access and every synchronisation operation in order to track the **happens-before** graph between threads. When two accesses to the same memory have no happens-before relation between them and at least one is a write, TSan reports a **data race** — with both stack traces, the access kinds (read vs write), and the location.

```
gcc -O1 -g3 -fno-omit-frame-pointer -fsanitize=thread ...
```

Cost. 5–15× CPU slowdown, 5–10× memory. Production-grade racy code becomes investigable; throughput-critical benchmarks risk to become unusable.

Like ASan it shifts what your program does — for race detection in particular, it is likely to **change** the schedule and miss intermittent races than to amplify them.

Cannot combine with ASan or MSan: all three rely on the same shadow-memory mechanism and want exclusive use of it. Run them in separate builds, on separate runs — the workflow in C8 already assumes this.

Comparing sanitizers & valgrind

Sanitizers instrument at **compile time** and then run on real hardware.

Valgrind interprets your program on a **synthetic CPU at run time**, requiring no recompilation.

	Sanitizers	Valgrind (Memcheck / Helgrind / ...)
Where the work happens	compile time + native run	interpretation at run time
Slowdown	~2× (ASan) · ~15× (TSan)	~30× (Memcheck); more for others
Recompile required	yes — pass -fsanitize=...	no — runs any binary
Buffer overflows, UAF	ASan, precise	Memcheck, precise
Uninitialised reads	MSan only (Clang + libc rebuilt)	Memcheck — gold standard
Undefined behaviour, broad	UBSan	not by default
Data races	TSan	Helgrind / DRD
Uninstrumented libraries	invisible to the sanitizer	all instrumented through the synthetic CPU

Sanitizers' behaviour

Environment Variable	Target Sanitizer	Typical Use Case
ASAN_OPTIONS	AddressSanitizer	Memory corruption, out-of-bounds, use-after-free.
UBSAN_OPTIONS	UndefinedBehaviorSanitizer	Integer overflow, null pointer dereference, alignment.
TSAN_OPTIONS	ThreadSanitizer	Data races, deadlocks, thread leaks in parallel execution.
MSAN_OPTIONS	MemorySanitizer	Use of uninitialized memory (requires compiling all dependencies).
LSAN_OPTIONS	LeakSanitizer	Memory leaks (often run implicitly as part of ASAN).
HWASAN_OPTIONS	HWAddressSanitizer	Hardware-assisted ASAN (primarily AArch64).
SANITIZER_OPTIONS	All Sanitizers	Base configuration shared across all active sanitizers.

It is possible to govern sanitizers behaviour by environmental variables, one per sanitizers class.
Examples:

```
# combine ASan and UBSan
gcc -O1 -g3 -fno-omit-frame-pointer \
  -fsanitize=address,undefined \
  -o prog source.c

# detect leaks at exit, stop on first error,
# raise SIGABRT so a core is produced
ASAN_OPTIONS="detect_leaks=1:\
  halt_on_error=1:\
  abort_on_error=1" ./prog

# print the stack trace (default is one line),
# and stop on the first UB rather than carry on
UBSAN_OPTIONS="print_stacktrace=1:\
  halt_on_error=1" ./prog
```

Some limitations of sanitizers

1

Sanitizers catch what their instrumentation watches

Code in dynamic libraries you linked but did not compile is invisible. In HPC that means: the MPI implementation, BLAS, FFTW, HDF5 — most of the dependency stack — runs uninstrumented. Bugs that live there, or that the application triggers through them (a wrong size passed to a library function that walks past your buffer), are unnoticed.

2

The instrumentation changes what your program does.

Redzones, larger allocations, occasional serialisation on hot paths — the program's timing is no longer the same. Timing-dependent bugs (a race that fires once in five at -O2) may appear, disappear, or shift.

3

Some allocator behaviour masks bugs by accident.

ASan does not return freed memory to the OS quickly; the freed bytes are typically still readable. A use-after-free that **reads** but does not write may give the right answer under ASan even though the diagnostic fires.

4

Suppressions are mandatory in real codebases.

`ASAN_OPTIONS=suppressions=file.txt`, `LSAN_OPTIONS=suppressions=...`, and a curated 'known leaks in \$LIB_VENDOR' list are part of every long-lived sanitizer workflow. This is not laziness; you cannot fix code you cannot rebuild.

5

A sanitizer says a bug exists; not that the program is correct.

Logical errors, intra-frame stack issues when frames live in registers, defined-but-wrong behaviour such as unsigned overflow — all pass through silently: « **absence of proof is not a proof of absence** »

Introducing threads

What still works

- every gdb command we have seen — info threads makes them thread-aware
- ASan and UBSan in their usual roles
- core files: a parallel program's core has all its threads in it
- rr records the schedule and replays it, deterministically

What is new

- races: two accesses to one variable, no happens-before, ≥ 1 write : use TSan
- deadlocks: a cycle in the lock-order graph — thread apply all bt
- OpenMP-only failure modes: forgotten reduction, missing barrier
- heisenbugs: observation perturbs the schedule
- the program is unreachable when you need it most — spin-and-attach

Concurrency bugs are not rare; they are **rarely observed**. A race fires on every run that contains it, in the formal sense — but you only **see** it when the schedule interleaves the threads the wrong way. A program can ship with a race that fires once a month in production, on the customer's machine, never in your tests. Hence the rule that organises the rest of this section: **do not rely on observation. Instrument.**

Introducing threads

One process, several threads; gdb knows about all of them. At any moment one thread is **current** — print, info locals, step act on it — and the others sit, suspended, until you continue. Switching is fast and reversible.

The commands

info threads

list every thread, with its ID, target tid, and where it currently sits

thread N

switch focus to thread N; prints, frames, stepping now act on it

thread apply all bt

a backtrace of every thread —

thread apply 1-4 print state

apply any command to a set of threads

break FUNC thread N

a breakpoint that fires only when thread N reaches FUNC

Stepping with fellows around

```
(gdb) set scheduler-locking on
(gdb) step
(gdb) set scheduler-locking off
```

on: only the current thread runs while you step — clean local view, time freezes for everyone else. **step**: the same, but the moment you **continue** the others run too. **off** (default): every thread can run between gdb stops; race bugs reproduce. Pick **on** while you walk a function alone; pick **off** to see the race fire.

Vocabulary. gdb's thread **ID** is gdb's own numbering (1, 2, 3, ...); the **target tid** shown next to it is the OS-level identifier (Linux: the gettid value), which is what **htop**, **perf**, and **ps -L** show.

Always check both columns of **info threads**

A race

from O_race_counter.c

```
long counter = 0;      /* shared */

int main(void) {
    #pragma omp parallel num_threads(2)
    {
        for (long i = 0; i < N; i++)
            counter++; /* THE RACE */
    }
    printf("counter = %ld\n", counter);
}
```

three runs of the same binary, three answers

```
$ ./race
counter = 1538421   (expected 2000000)
$ ./race
counter = 1872003   (expected 2000000)
$ ./race
counter = 1145967   (expected 2000000)
$ ./race # the lucky one
counter = 2000000   (expected 2000000)
```

printf is likely useless to debug a race. Printf takes an internal lock on stdout and serialises the threads — the race window closes and you stop seeing the bug.
See later

The race detector

TSan instruments every memory access and every synchronisation operation in order to build the **happens-before** graph between threads. Two accesses to the same memory with no happens-before relation between them, and at least one a write, is a **race** — and TSan reports it, with both stack traces.

```
gcc -O1 -g3 -fopenmp -fno-omit-frame-pointer -fsanitize=thread ...
```

TSan on 0_race_counter.c — the OpenMP region is outlined as main._omp_fn.0

=====

WARNING: ThreadSanitizer: data race (pid=12345)

Write of size 8 at 0x... by main thread:

#0 `main._omp_fn.0` `0_race_counter.c:113`

#1 `GOMP_parallel` `libgomp.so.1`

Previous read of size 8 at 0x... by thread T1:

#0 `main._omp_fn.0` `0_race_counter.c:113`

#1 `libgomp.so.1`

Location is global 'counter' of size 8 at 0x...

Thread T1 (tid=..., running) created by main thread at:

#0 `pthread_create` `(tsan interceptor)`

#1 `libgomp.so.1`

Cost. 5–15× CPU, 5–10× memory. Significant but bearable on tests; unusable on throughput benchmarks. **Cannot combine** with ASan / MSan: same shadow-memory machinery, exclusive. Build them separately.

TSan detects races on the runs you **ran**. Absence of a TSan report is not a proof of correctness — it is evidence that **this schedule** did not exercise a race. Run many times, on many machines, under load; **--chaos** of rr (B15) helps shake out the shy ones.

The race detector

TSan instruments every memory access and every synchronisation operation in order to build the **happens-before** graph between threads. Two accesses to the same memory with no happens-before relation between them, and at least one a write, is a **race** — and TSan reports it, with both stack traces.

```
gcc -O1 -g3 -fopenmp -fno-omit-frame-pointer -fsanitize=thread ...
```

TSan on 0_race_counter.c — the OpenMP region is outlined as main._omp_fn.0

=====

WARNING: ThreadSanitizer: data race (pid=12345)

Write of size 8 at 0x... by main thread:

#0 main._omp_fn.0 0_race_counter.c:113

#1 GOMP_parallel libgomp.so.1

Previous read of size 8 at 0x... by thread T1:

#0 main._omp_fn.0 0_race_counter.c:113

#1 libgomp.so.1

Location is global 'counter' of size 8 at 0x...

Thread T1 (tid=..., running) created by main thread at:

#0 pthread_create (tsan interceptor)

#1 libgomp.so.1

Cost. 5–15× CPU, 5–10× memory. Significant but bearable on tests; unusable on throughput benchmarks. **Cannot combine** with ASan / MSan: same shadow-memory machinery, exclusive. Build them separately.

TSan detects races on the runs you **ran**. Absence of a TSan report is not a proof of correctness — it is evidence that **this schedule** did not exercise a race. Run many times, on many machines, under load; **--chaos** of rr (B15) helps shake out the shy ones.

A deadlock

Symptom. The program is alive but does no work — no output, no CPU, no progress. **top** shows zero CPU; **ps** still lists the process; Ctrl-C is the only way out.

Cause. Two OpenMP sections, two locks, opposite acquisition orders (**1_deadlock.c**). Section 1 holds A, waits on B; section 2 holds B, waits on A. The lock-order graph has a cycle.

Hint: TSan detects **lock-order inversions** even on runs that **do not** actually deadlock — it tracks the order of acquisitions across the run and reports any cycle in the graph. This is how you find a deadlock **before** the production hang. **1_deadlock.c**, exercise 2.

```
$ ./deadlock &
$ gdb -p $!
(gdb) thread apply all bt
```

libgomp: both sections run inside the outlined function main._omp_fn.0

Thread 3 (section 2):

```
#0  __lll_lock_wait ()
#1  gomp_mutex_lock_slow ()
#2  omp_set_lock ()
#3  main._omp_fn.0  1_deadlock.c:123  <holds B, waits A>
```

Thread 2 (section 1):

```
#0  __lll_lock_wait ()
#1  gomp_mutex_lock_slow ()
#2  omp_set_lock ()
#3  main._omp_fn.0  1_deadlock.c:108  <holds A, waits B>
>>> cycle:  A <- thread 2 -> B <- thread 3 -> A
```

Clerical error

missing reduction --> `3_reduction_wrong.c`

missing barrier --> `2_missing_barrier.c`

The heisenbugs

printf takes an internal lock on the FILE* (POSIX's **flockfile**, invisible to the caller). Every iteration now serialises through that lock — only one thread sits between load and store at a time.

The race window has been closed by the **I/O lock**, accidentally and silently.

The Spin-Wait trick

that's all, have fun

